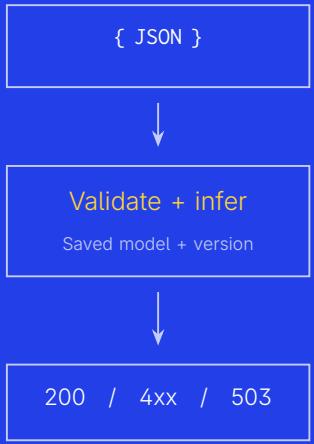
WEEK 2 / LAB 04

MLOps: Machine Learning Operations

Prediction API

Input validation & containerised tests

Serve the evaluated bike-demand model.
Verify its behaviour before deployment.



Show three observable service behaviours

Completion means checked service behaviour, not just a running process.

01	Serve	Load the evaluated model; return its actual version.
02	Validate	Reject missing fields and invalid types or ranges.
03	Verify	Test normal and invalid requests plus a missing artifact.

Continue from the Lab 3 handover

Use the same team repository, trusted saved model and approved environment.

01

Artifact

Evaluated model and matching metadata.

No retraining to serve.

02

Environment

Lab 3 Dockerfile and approved dependency lock.

API and test tools included.

03

Access

Docker works; a local port and reviewer are available.

Ask early about blockers.

If a prerequisite is missing, use the approved recovery route.

Build a contract, then try to break it

Check prerequisites, build, test and run the image together.

5 min	20 min	20 min	15 min	5 min
Preflight	**Build**	**Test**	**Container**	**Review**
Confirm the handover.	API and validation.	Three behaviour families.	Tests and HTTP smoke check.	Evidence and next action.

Rotate the driver; review the code and evidence together.

Success and failure need different responses

Two endpoints share one model-availability state.

REQUEST / STATE	HTTP STATUS	OBSERVABLE RESULT
GET /health	200 or 503	Usable model or unavailable service.
POST /predict	200	Finite non-negative number + version.
Invalid input / missing model	4xx / 503	Clear error; never a fake prediction.

A valid request with an unavailable model receives 503.

Use the twelve fields from training

One flat JSON object; keep source category codes and normalised weather.

Calendar: seven integers

`season`: 1–4 `yr`: 0–1
`mnth`: 1–12 `hr`: 0–23
`holiday`: 0–1 `weekday`: 0–6
`workingday`: 0–1

Weather: five values

`weathersit`: integer 1–4
`temp`, `atemp`, `hum`, `windspeed`:
finite numbers in [0, 1].

Not Celsius, percentages or weather labels.

Do not accept cnt, casual or registered as predictors.

Return a prediction and the loaded version

Example request: save a valid fixture for your tests.

REQUEST JSON – ALL TWELVE FIELDS

```
{
  "season": 2, "yr": 1, "mnth": 5,
  "hr": 8, "holiday": 0, "weekday": 2,
  "workingday": 1, "weathersit": 1,
  "temp": 0.50, "atemp": 0.48,
  "hum": 0.60, "windspeed": 0.20
}
```

Response checks

prediction

JSON number, finite, non-negative.

model_version

Version of the model actually loaded.

A hard-coded label is not evidence of serving identity.

Validate types and ranges before inference

Pydantic v2 example: two fields shown; implement all twelve.

REQUEST SCHEMA EXCERPT

```python
from pydantic import BaseModel, ConfigDict, Field

class RequestExcerpt(BaseModel):
    model_config = ConfigDict(extra="forbid")
    hr: int = Field(ge=0, le=23, strict=True)
    hum: float = Field(
        ge=0, le=1, strict=True,
        allow_inf_nan=False,
    )
```

Three choices

Required fields

No silent defaults for missing inputs.

Strict numbers

Reject invalid types and non-finite values.

Reject extras here; document the policy.

Serving reuses the evaluated model

OFFLINE – LABS 2–3

Train and evaluate

Fit preprocessing on training rows.

Evaluate; save model and metadata.

Record the run and configuration.

ONLINE – LAB 4

Load and predict

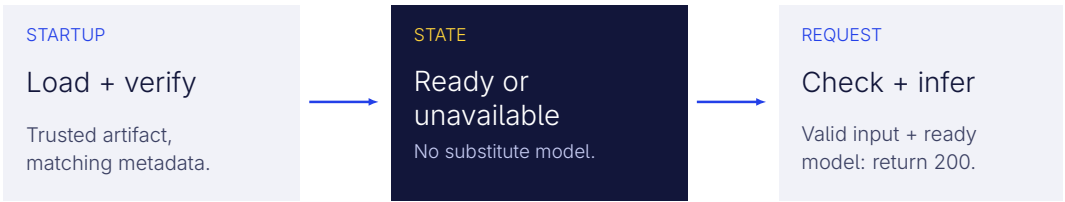
Reuse fitted preprocessing unchanged.

Verify the saved artifact and its identity.

Validate requests; infer without fitting.

Load once; make availability explicit

The model and its version belong to the same application state.



Expected load failure: /health and valid /predict return 503.

A missing artifact must not trigger training or a dummy result.

TEAM ACTIVITY A

Implement the service without retraining

Add the request schema and endpoints.
Load the saved model during startup.

Check your result

Valid input returns 200 + actual version.
OpenAPI shows the twelve-field contract.

20 MINUTES – BUILD

1. Schema
2. Load + availability
3. Health + predict

Rotate driver and reviewer.
Keep actual evidence.

PART 2 – BEHAVIOUR

Make failure observable

A green happy-path test is not enough to trust the interface.

TEST THREE FAMILIES

- Normal input
- Invalid input
- Missing artifact

Tests must cover three behaviour families

Start each case in the state it is meant to test.

TEST FAMILY	SETUP	ASSERTION
Normal	Loaded model + valid payload	200; finite value; correct version.
Invalid	Loaded model + one bad field	Clear 4xx; no successful prediction.
Unavailable	Fresh app + absent artifact	Health 503; valid prediction 503.

A normal-input test cannot prove the missing-model branch.

Check the value and the model identity

Pytest example: use the trusted evaluated artifact in your fixtures.

NORMAL-INPUT TEST EXCERPT

```python
import math


def test_predict(client, valid_request, artifact_version):
    response = client.post("/predict", json=valid_request)
    assert response.status_code == 200
    body = response.json()
    value = body["prediction"]
    assert type(value) in (int, float)
    assert math.isfinite(value) and value >= 0
    assert body["model_version"] == artifact_version
```

Not just 200

Numeric output

No strings, booleans, NaN or infinity.

Known version

Expected identity comes from the artifact record.

Change one field to test validation

Start from the valid fixture; FastAPI's default validation response is 422.

CASE	MUTATION	EXPECTED RESULT
Missing field	Remove `hr`.	422; field is required.
Wrong type	Set `hr` to `"eight"`.	422; integer expected.
Out of range	Set `hr=24` or `hum=60`.	422; reject invalid scale.

Keep valid boundary values valid: hr=0 and hr=23.

A fresh app reveals a missing artifact

MISSING-ARTIFACT TEST EXAMPLE

```python
from fastapi.testclient import TestClient
from bike_demand.api import create_app

def test_missing_artifact(tmp_path, valid_request):
    missing = tmp_path / "missing.joblib"
    fresh_app = create_app(model_path=missing)
    with TestClient(fresh_app) as client:
        assert client.get("/health").status_code == 503
        response = client.post("/predict", json=valid_request)
    assert response.status_code == 503
    assert "prediction" not in response.json()
```

Fresh startup

No cached model from a previous test.

Run lifespan

The `with` block triggers startup and shutdown.

Adapt `create_app` to your implementation.

TEAM ACTIVITY B

Prove the three behaviour families

Add normal and invalid-input tests.
Start a fresh app with a missing artifact.

Check your result

All three families run; failed assertions lead to fixes, not skipped tests.

20 MINUTES – TEST

1. Valid fixture
2. One-field mutations
3. Fresh missing model

Rotate driver and reviewer.
Keep actual evidence.

Test the environment inside the image

Extend the Lab 3 image; do not silently depend on files from your laptop.

01

Code + lock

API source and the approved dependency set.

No arbitrary upgrades.

02

Test tools

pytest, HTTPX and fixtures are included.

The test command can run.

03

Model files

Trusted artifact and its matching metadata.

Allow only the reviewed export.

No host source mount in this check: test the built image.

Rebuild, then test inside the image

Run from your team project root, using the Lab 3 environment.

CONTAINER TEST COMMANDS

```
docker build -t bike-demand:lab04 .
docker image inspect --format '{{.Id}}' bike-demand:lab04
docker run --rm --network none --entrypoint .venv/bin/python \
  bike-demand:lab04 -m pytest -q
```

No host bind mount; the container's own files are exercised.

Capture evidence

Source commit
Image identity
Real test summary

A zero exit status is necessary, not a quality score.

Local pytest passing does not prove container tests ran.

Check HTTP through the published port

Listen inside the container; publish the port only to your local host.

TERMINAL A – FOREGROUND SERVICE, SAME IMAGE

```
docker run --rm --name bike-lab04 \
  -p 127.0.0.1:8000:8000 \
  --entrypoint .venv/bin/python bike-demand:lab04 \
  -m uvicorn bike_demand.api:app \
  --host 0.0.0.0 --port 8000
```

Stop with Ctrl+C after the check; `--rm` removes the container.

Terminal B

1. Health

Check port 8000 and `/health`.

2. Prediction

Open `/docs`; send the valid fixture. Check value + version.

TestClient tests do not verify Docker port publishing.

Diagnose the failing layer first

Match the symptom to the first check; change only the failing layer.

SYMPTOM	FIRST CHECK	AVOID
503 with valid input	Artifact path; load diagnostic.	Training a fallback model.
422 for the fixture	Required fields; input ranges.	Weakening range checks.
Connection refused	Process logs and port mapping.	Treating this as a model error.

Fix the cause; retain the assertion that exposed it.

TEAM ACTIVITY C

Verify the image another member will run

Rebuild it and run the tests inside.
Then send one real HTTP prediction.

Check your result

Record actual image and model identities, test results and the smoke response.

15 MINUTES – CONTAINER

1. Rebuild
2. Container pytest
3. HTTP smoke check

Rotate driver and reviewer.
Keep actual evidence.

Connect the code to the tested model

Link the source, model and execution evidence in your team repository.

01	Source	Commit + configuration + run instructions.
02	Model	Trusted artifact + matching loaded model version.
03	Execution	Image identity + real tests + HTTP smoke response.

One Week 2 checkpoint; one personal report

TEAM PROJECT REPOSITORY

Checkpoint m2

Review Labs 3–4;
tag the intended commit.

Short note: lock and rerun; API and container tests.

Ungraded checkpoint.
No weekly Release.

PRIVATE PERSONAL REPOSITORY

reports/week-2.md

Explain your own contribution across Labs 3–4.

Commit directly to personal main; no report PR or CI.

10% of course; Sunday 23:59 at the published date/time zone.

BEFORE YOU CALL LAB 4 COMPLETE

A usable service tells the truth when it fails

01 Evaluated artifact loaded; no training in the service.

02 Normal, invalid and unavailable behaviours verified.

03 Container tests and a real HTTP smoke check recorded.

Next, Lab 5: local deployment, structured logs and serving-version checks.